

Queues: First In, First Out

Queue ADT, Terminology & Array Representation

Unit II • Lecture 11 (first lecture of Unit II)

the FIFO discipline • front & rear • enqueue & dequeue • array implementation

← OUT (front)

10

20

30

IN (rear) ←

From LIFO to FIFO: what changes?

Stack (Unit I)

One end. Last In, First Out.

A pile of plates — the newest plate leaves first.

Queue (Unit II)

Two ends. First In, First Out.

A ticket line — the person waiting longest is served first.

- Insertion happens only at the REAR; deletion happens only at the FRONT.
- Two access points means two markers to manage — the stack needed only top.
- Same course pattern as Unit I: define the ADT first, then build it on an array, then on linked nodes.

The Queue ADT: a fair waiting line



- A linear structure where insertion and deletion happen at OPPOSITE ends.
- The element waiting longest is always served next — fairness by design.
- Real queues everywhere: printer jobs, CPU scheduling, call-centre lines, keyboard buffer.

Definition to note: a queue is a linear list in which insertions are made at one end (rear) and deletions at the other end (front) — First In, First Out (FIFO).

Queue ADT — the operation set

Operation	Purpose	Time
<code>enqueue(x)</code>	Insert x at the REAR	$O(1)$
<code>dequeue()</code>	Remove and return the FRONT element	$O(1)$
<code>peek()</code>	Read the front element without removing it	$O(1)$
<code>isEmpty()</code>	Check whether the queue has no elements	$O(1)$
<code>isFull()</code>	Check whether the queue is full (array only)	$O(1)$
<code>display()</code>	Print all elements, front to rear	$O(n)$

Same interface shape as the stack — push/pop became enqueue/dequeue, and both ends now have names.

Array representation: two markers instead of one

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1;    /* serving end */
int rear  = -1;    /* joining end */
```

- front is the index of the element to serve next.
- rear is the index of the most recently joined element.
- Both -1 means the queue is empty — the same sentinel idea as the stack's top = -1.
- Elements always occupy the slice queue[front] .. queue[rear].

The new difficulty

The stack grew and shrank at ONE end, so one index was enough. A queue moves at BOTH ends — front chases rear across the array.

isEmpty() and isFull() — the guard functions

```
int isEmpty(void)
{
    return front == -1;
}
```

```
int isFull(void)
{
    return rear == MAX - 1;
}
```

Underflow guard

dequeue on an empty queue is queue underflow — checked before every removal.

Overflow guard — with a flaw

rear == MAX - 1 says "full". But is the array really full when front has moved forward? Hold that thought.

enqueue() — join at the rear

```
void enqueue(int x)
{
    if (isFull())
    {
        printf("Queue Overflow\n");
        return;
    }

    if (front == -1)
        front = 0;    /* first element */

    rear = rear + 1;
    queue[rear] = x;
}
```

- Guard first: overflow check, exactly as push did.
- The special line: the FIRST enqueue must also wake front up from -1 to 0 — otherwise the queue holds data that front cannot see.
- Then the familiar pair: advance rear, store at rear.

dequeue() — serve from the front

```
int dequeue(void)
{
    if (isEmpty())
    {
        printf("Queue Underflow\n");
        return -1;
    }

    int x = queue[front];

    if (front == rear)
    {
        front = -1;    /* last element */
        rear = -1;     /* reset both */
    }
    else
        front = front + 1;

    return x;
}
```

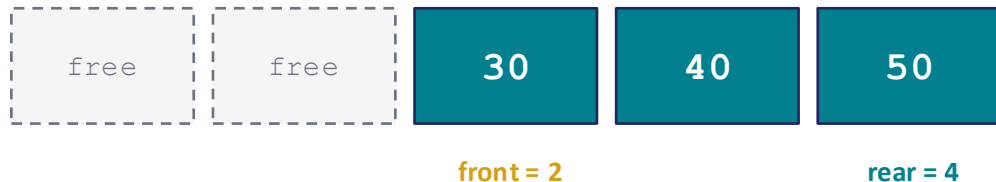
- Save the front element, then move the marker — the value is not erased, just left behind.
- The special case mirrors enqueue's: removing the LAST element resets both markers to -1, restoring the true empty state.
- Without the reset, front would overtake rear — a state that means nothing.

Trace: watch front chase rear

Operation	Queue [0..4]	front	rear
start	- - - - -	-1	-1
enqueue (10)	10 - - - -	0	0
enqueue (20)	10 20 - - -	0	1
enqueue (30)	10 20 30 - -	0	2
dequeue () → 10	· 20 30 - -	1	2
dequeue () → 20	· · 30 - -	2	2
enqueue (40)	· · 30 40 -	2	3
enqueue (50)	· · 30 40 50	2	4

· marks a slot whose value was served — the data is still there, but front has moved past it.

The flaw revealed: false overflow



`enqueue(60)` now says "Queue Overflow"

`rear == MAX - 1`, so `isFull()` answers yes — while two slots stand empty on the left. The queue is "full" and 40% empty at the same time.

- `front` and `rear` only ever move right; served slots are never reused.
- This is FALSE overflow — the defining defect of the linear array queue.
- Fix preview (Lecture 13): let the indices wrap around to slot 0 — the circular queue.

peek() and display()

```
int peek(void)
{
    if (isEmpty())
    {
        printf("Queue is Empty\n");
        return -1;
    }
    return queue[front];
}
```

```
void display(void)
{
    if (isEmpty())
    {
        printf("Queue is Empty\n");
        return;
    }
    printf("FRONT -> ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("<- REAR\n");
}
```

peek reads queue[front] — the next to be served, not the last to arrive. display walks the live slice front..rear in $O(n)$.

Stack vs Queue — side by side

Aspect	Stack	Queue
Discipline	LIFO — last in, first out	FIFO — first in, first out
Access ends	One (top)	Two (front and rear)
Insert / delete	push / pop, both at top	enqueue at rear / dequeue at front
Markers (array)	top = -1 when empty	front = rear = -1 when empty
Typical uses	Undo, recursion, expressions	Scheduling, buffering, BFS

Where queues run your computer

Printer spooler

Documents print in the order submitted — the canonical FIFO.

CPU scheduling

Ready processes wait in a queue for their turn on the processor.

Keyboard buffer

Keystrokes are queued so none are lost while the CPU is busy.

BFS traversal (Unit V)

Breadth-first search explores a graph level by level using a queue.

The queue: fairness as a data structure

- A queue inserts at the rear and deletes at the front — FIFO, the opposite discipline to the stack.
- Array version: two markers, front and rear, both -1 when empty; elements live in the slice front..rear.
- Two special cases: the first enqueue wakes front to 0; removing the last element resets both to -1.
- The linear queue's defect is false overflow — "full" at rear == MAX - 1 while served slots sit empty on the left.

EXIT TICKET With MAX = 4, trace: enqueue(5), enqueue(8), dequeue, enqueue(2), enqueue(7), dequeue, enqueue(9). Show front, rear and the array after each step. Where does false overflow strike?